

LINQ SequenceEqual Method in C#

In this article, I will discuss the **LINQ SequenceEqual Method in C#** with Examples. Please read our previous article, discussing the [LINQ DefaultIfEmpty Method in C#](#) with Examples. The SequenceEqual method in LINQ determines whether two sequences are equal by pairing their elements. It checks if the two sequences have the same number of elements and if each element in one sequence equals the corresponding element in the other.

LINQ SequenceEqual Method in C#

The LINQ SequenceEqual Method in C# checks whether two sequences are equal. If two sequences are equal, it returns true; otherwise, it returns false. Two sequences are considered to be equal when both the sequences have the same number of elements, as well as the same values, and should be present in the same order. There are two overloaded versions available for this SequenceEqual Method in LINQ. They are as follows.

```
public static bool SequenceEqual<TSource>(this IEnumerable<TSource> first, IEnumerable<TSource> second);  
public static bool SequenceEqual<TSource>(this IEnumerable<TSource> first, IEnumerable<TSource> second,  
IEqualityComparer<TSource> comparer);
```

As you can see, the second overloaded version takes an extra IEqualityComparer parameter. So, whenever you don't want the default comparer to be used by the LINQ SequenceEqual Method, you need to use this overloaded version to use your own comparer.

Key points to note about SequenceEqual:

- It compares the elements in both sequences one by one, from the beginning to the end.
- If the two sequences' lengths are different or any element in the first sequence does not match the corresponding element in the second sequence, the result will be false.
- The sequences must contain elements of the same type for the comparison to work correctly.

The SequenceEqual method is useful when you need to verify if two sequences are identical in terms of their elements and order. It can be used for equality checks in various scenarios, such as verifying data integrity, comparing results, or checking for matching sequences in unit tests.

Example to Understand LINQ SequenceEqual Method in C#:

Let us see an example to understand the LINQ SequenceEqual Method in C# using both method and query syntax. In the following example, we created two sequences to store the cities. As you can see, both collections contain the same number of elements. Further, if you notice, all the elements are in the same order in both collections. So here, the sequences are equal, and the SequenceEqual method will return true. There is no such operator called SequenceEqual available in LINQ to write the query syntax, so, in that case, we can go with method syntax.

```
using System.Linq;  
using System;  
using System.Collections.Generic;
```

```

namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<string> cityList1 = new List<string> { "Delhi", "Mumbai", "Hydrabad" };
            //Collection 2 or Sequence 2
            List<string> cityList2 = new List<string> { "Delhi", "Mumbai", "Hydrabad" };
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = cityList1.SequenceEqual(cityList2);
            //Using Query Syntax
            bool IsEqualQS = (from city in cityList1
                              select city).SequenceEqual(cityList2);
            //Printing the Result
            Console.WriteLine(IsEqualQS);
            Console.ReadLine();
        }
    }
}

```

Output: True

Now, modify the example as follows. Here, we have the same cities but with different cases. The default compare, used by the SequenceEqual method, is case-sensitive. So, the below example returns false as the values are case-sensitive.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<string> cityList1 = new List<string> { "DELHI", "mumbai", "Hydrabad" };
            //Collection 2 or Sequence 2
            List<string> cityList2 = new List<string> { "delhi", "MUMBAI", "Hydrabad" };
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = cityList1.SequenceEqual(cityList2);
            //Using Query Syntax
            bool IsEqualQS = (from city in cityList1
                              select city).SequenceEqual(cityList2);
            //Printing the Result
            Console.WriteLine(IsEqualQS);
            Console.ReadLine();
        }
    }
}

```

```

    }
}
}

```

Output: False

If you want the comparison to be case-insensitive, you need to use the other overloaded version of the SequenceEqual() method, which takes IEqualityComparer as a parameter. So, modify the Example as follows to use the second overloaded version of the SequenceEqual() method, which takes IEqualityComparer. Here, you can see we are passing StringComparison.OrdinalIgnoreCase is the second parameter to the SequenceEqual() method. The StringComparison class implements the IEqualityComparer interface.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<string> cityList1 = new List<string> { "DELHI", "mumbai", "Hyd" };
            //Collection 2 or Sequence 2
            List<string> cityList2 = new List<string> { "delhi", "MUMBAI", "Hyd" };
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = cityList1.SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Using Query Syntax
            bool IsEqualQS = (from city in cityList1
                             select city).SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Printing the Result
            Console.WriteLine(IsEqualMS);
            Console.ReadLine();
        }
    }
}

```

Output: True

Note: If you go to the definition of StringComparison class, you will see that this class implements the IEqualityComparer interface.

What Happens if the Elements are not present in the same order?

If the Elements are not present in the same order in both sequences, then the SequenceEqual method will return False. In the following example, the SequenceEqual method returns false. This is because the

elements are not in the same order in both sequences.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<string> cityList1 = new List<string> { "Delhi", "Mumbai", "Hyderabad" };
            //Collection 2 or Sequence 2
            List<string> cityList2 = new List<string> { "Delhi", "Hyderabad", "Mumbai" };
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = cityList1.SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Using Query Syntax
            bool IsEqualQS = (from city in cityList1
                             select city).SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Printing the Result
            Console.WriteLine(IsEqualMS);
            Console.ReadLine();
        }
    }
}
```

Output: **False**

How Can We Solve the Above Problem?

If sequences contain the same data but in a different order, and you want the SequenceEqual method to return true, then, first, you need to sort the data either in ascending or descending order in both the sequences and then only apply the LINQ SequenceEqual method which is shown in the below example.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<string> cityList1 = new List<string> { "Delhi", "Mumbai", "Hyderabad" };
            //Collection 2 or Sequence 2
            List<string> cityList2 = new List<string> { "Delhi", "Hyderabad", "Mumbai" };
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = cityList1.SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Using Query Syntax
            bool IsEqualQS = (from city in cityList1
                             select city).SequenceEqual(cityList2,
StringComparer.OrdinalIgnoreCase);
            //Printing the Result
            Console.WriteLine(IsEqualMS);
            Console.ReadLine();
        }
    }
}
```

```

        List<string> cityList2 = new List<string> { "Delhi", "Hyderabad",
        //Checking if both Sequences are Equal or not
        //Using Method Syntax
        bool IsEqualMS = cityList1.OrderBy(city =>
city).SequenceEqual(cityList2.OrderBy(city => city), StringComparer.OrdinalIgr
        //Using Query Syntax
        bool IsEqualQS = (from city in cityList1.OrderBy(city => city)
                        select city).SequenceEqual(cityList2.OrderBy(city
StringComparer.OrdinalIgnoreCase);
        //Printing the Result
        Console.WriteLine(IsEqualMS);
        Console.ReadLine();
    }
}
}

```

Output: **True**

Working with Complex Type using LINQ SequenceEqual Method:

Let us understand how to work with the LINQ SequenceEqual with Complex Data type in C#. To understand this, we will work with the following Student class. So, create a class file named **Student.cs** and copy and paste the following code. Here, you can see we have created the Student class with two properties, i.e., ID and Name. Then we have also created two methods, i.e., GetStudents1 and GetStudents2. Further, if you notice, these two methods will return the same data.

```

using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Student> GetStudents1()
        {
            List<Student> listStudents = new List<Student>()
            {
                new Student{ID= 101,Name = "Preety"},
                new Student{ID= 102,Name = "Priyanka"}
            };
            return listStudents;
        }
        public static List<Student> GetStudents2()
        {
            List<Student> listStudents = new List<Student>()
            {
                new Student{ID= 101,Name = "Preety"},
                new Student{ID= 102,Name = "Priyanka"}
            };
            return listStudents;
        }
    }
}

```

```

    }
}
}

```

Now, what we will do is we will check whether both the sequences that are returned by the GetStudents1 and GetStudents2 methods are the same or not using the LINQ SequenceEqual method. So, modify the Main method of the Program class as shown below.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<Student> StudentList1 = Student.GetStudents1();
            //Collection 2 or Sequence 2
            List<Student> StudentList2 = Student.GetStudents2();
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            bool IsEqualMS = StudentList1.SequenceEqual(StudentList2);
            //Using Query Syntax
            bool IsEqualQS = (from std in StudentList1
                             select std).SequenceEqual(StudentList2);
            //Printing the Result
            Console.WriteLine(IsEqualMS);
            Console.ReadLine();
        }
    }
}

```

Output: False

Even though both sequences contain the same data, we are getting the result as False. This is because when working with complex types, the default comparer used by the SequenceEqual method will only check if the object references are equal or not. It is not checking the object values; rather, it is checking the object references, and references are different in this case, and hence it returns False.

How to Solve the Above Problem?

Solving the Above Problem means ensuring the SequenceEqual method goes for value equality rather than reference equality, i.e., we need to tell the SequenceEqual method to check the individual object values rather than object reference addresses. There are many ways we can solve the above problem. Some of them are as follows.

1. Creating Custom Comparer. We need to use the other overloaded version of the SequenceEqual method to pass a custom class that implements the **IEqualityComparer** interface.

2. We must Project the properties into a new anonymous type, which already overrides **Equals()** and **GetHashCode()** methods.
3. We must override the Equals() and GetHashCode() methods in the Student class.
4. By Implementing **IEquatable<T>** interface.

Creating Custom StudentComparer Class:

Create a class file named **StudentComparer.cs** and copy and paste the following code. As you can see, this class implements the IEqualityComparer interface and provides the implementation for the Equals and GetHashCode methods.

```
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    public class StudentComparer : IEqualityComparer<Student>
    {
        public bool Equals(Student x, Student y)
        {
            return x.ID == y.ID && x.Name == y.Name;
        }
        public int GetHashCode(Student obj)
        {
            return obj.ID.GetHashCode() ^ obj.Name.GetHashCode();
        }
    }
}
```

Next, modify the Main method of the Program class as follows. Now, we need to create an instance of the StudentComparer class, and then we need to pass that instance to the SequenceEqual method, shown in the code below. With these changes, the compare, which we pass to the SequenceEqual method, will be used to check the equality, and this will use the individual object values, not the object reference.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<Student> StudentList1 = Student.GetStudents1();
            //Collection 2 or Sequence 2
            List<Student> StudentList2 = Student.GetStudents2();
            //Creating an Instance of StudentComparer
            StudentComparer studentComparer = new StudentComparer();
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            //Passing the StudentComparer Instance to the SequenceEqual method
```

```

        bool IsEqualMS = StudentList1.SequenceEqual(StudentList2, studentC
//Using Query Syntax
//Passing the StudentComparer Instance to the SequenceEqual method
        bool IsEqualQS = (from std in StudentList1
                        select std).SequenceEqual(StudentList2, studentC
//Printing the Result
        Console.WriteLine(IsEqualMS);
        Console.ReadLine();
    }
}
}

```

Output: **True**

Using Anonymous Type with LINQ SequenceEqual Method:

In this approach, we must select all the individual properties to an anonymous type. The following program does exactly the same thing. In the below example, we are projecting the data into an anonymous type.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<Student> StudentList1 = Student.GetStudents1();
            //Collection 1 or Sequence 1
            List<Student> StudentList2 = Student.GetStudents2();

            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            //Projecting the Individual Properties to an Anonymous Type
            bool IsEqualMS = StudentList1
                .Select(std => new { std.ID, std.Name })
                .SequenceEqual(StudentList2.Select(std => new { s
std.Name }));
            //Using Query Syntax
            //Projecting the Individual Properties to an Anonymous Type
            bool IsEqualQS = (from std in StudentList1
                            select new { std.ID, std.Name })
                .SequenceEqual(StudentList2.Select( std => new {
std.Name }));
            //Printing the Result
            Console.WriteLine(IsEqualQS);
            Console.ReadLine();
        }
    }
}

```

```
}  
}
```

Output: **True**

Overriding Equals() and GetHashCode() Methods within the Student Class

By default, any type (predefined or user-defined) is inherited from the Object class. That means the Student class is also inherited from the Object class. And we also know that the Object class provides virtual methods such as Equals() and GetHashCode(). So, we need to override the Object class's Equals() and GetHashCode() methods within the Student class. So, modify the Student class as shown below. We are overriding the Equals() and GetHashCode() methods here.

```
using System.Collections.Generic;  
namespace LINQSequenceEqualDemo  
{  
    public class Student  
    {  
        public int ID { get; set; }  
        public string Name { get; set; }  
        public static List<Student> GetStudents1()  
        {  
            List<Student> listStudents = new List<Student>()  
            {  
                new Student{ID= 101,Name = "Preety"},  
                new Student{ID= 102,Name = "Priyanka"}  
            };  
            return listStudents;  
        }  
        public static List<Student> GetStudents2()  
        {  
            List<Student> listStudents = new List<Student>()  
            {  
                new Student{ID= 101,Name = "Preety"},  
                new Student{ID= 102,Name = "Priyanka"}  
            };  
            return listStudents;  
        }  
        //Overriding the Object class Equals Method  
        public override bool Equals(object x)  
        {  
            return this.ID == ((Student)x).ID && this.Name == ((Student)x).Name;  
        }  
        //Overriding the Object class GetHashCode Method  
        public override int GetHashCode()  
        {  
            return this.ID.GetHashCode() ^ this.Name.GetHashCode();  
        }  
    }  
}
```

With the above changes in place, now modify the Main method of the Program class as shown below. Now, we need to use the overloaded version of the LINQ SequenceEqual method, which does not take any parameter.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<Student> StudentList1 = Student.GetStudents1();
            //Collection 1 or Sequence 1
            List<Student> StudentList2 = Student.GetStudents2();
            //Checking if both Sequences are Equal or not
            //Using Method Syntax
            //Use the Overloaded version of the SequenceEqual method which does not take any
            //comparer parameter
            bool IsEqualMS = StudentList1.SequenceEqual(StudentList2);
            //Using Query Syntax
            //Use the Overloaded version of the SequenceEqual method which does not take any
            //comparer parameter
            bool IsEqualQS = (from std in StudentList1
                              select std)
                              .SequenceEqual(StudentList2);
            //Printing the Result
            Console.WriteLine(IsEqualQS);
            Console.ReadLine();
        }
    }
}
```

Output: **True**

Implementing IEquatable<T> Interface in Student Class.

In this approach, we need to implement the IEquatable<T> Interface in Student Class, and we need to implement the Equals Method of the IEquatable<T> Interface. We also need to override the GetHashCode method of the Object class. So, modify the Student class as shown below.

```
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    public class Student : IEquatable<Student>
    {
        public int ID { get; set; }
    }
}
```

```

public string Name { get; set; }
public static List<Student> GetStudents1()
{
    List<Student> listStudents = new List<Student>()
    {
        new Student{ID= 101,Name = "Preety"},
        new Student{ID= 102,Name = "Priyanka"}
    };
    return listStudents;
}
public static List<Student> GetStudents2()
{
    List<Student> listStudents = new List<Student>()
    {
        new Student{ID= 101,Name = "Preety"},
        new Student{ID= 102,Name = "Priyanka"}
    };
    return listStudents;
}
//Implementing the Equals Method of IEquatable Interface
public bool Equals(Student other)
{
    return this.ID.Equals(other.ID) && this.Name.Equals(other.Name);
}
//Overriding the Object class GetHashCode Method
public override int GetHashCode()
{
    return this.ID.GetHashCode() ^ this.Name.GetHashCode();
}
}
}

```

As you can see, here we have done two things. First, we implement the Equals method of the IEquatable interface, and then we override the GetHashCode method of the Object class. With the above changes in place, now modify the Main Method of the Program class as shown below. Again, here, we need to use the overloaded version of the LINQ SequenceEqual method, which does not take any parameter.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQSequenceEqualDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Collection 1 or Sequence 1
            List<Student> StudentList1 = Student.GetStudents1();
            //Collection 2 or Sequence 2
            List<Student> StudentList2 = Student.GetStudents2();
            //Checking if both Sequences are Equal or not

```

```

        //Using Method Syntax
        //Use the Overloaded version of the SequenceEqual method which does not require a
        comparer parameter
        bool IsEqualMS = StudentList1.SequenceEqual(StudentList2);
        //Using Query Syntax
        //Use the Overloaded version of the SequenceEqual method which does not require a
        comparer parameter
        bool IsEqualQS = (from std in StudentList1
                           select std)
                           .SequenceEqual(StudentList2);
        //Printing the Result
        Console.WriteLine(IsEqualQS);
        Console.ReadLine();
    }
}
}

```

Output: **True**

Important Points to Consider

- **Element Comparer:** SequenceEqual uses the default equality comparer for the type of elements in the sequence by default. If you need to compare elements based on a specific property or use custom logic, you may need to provide an `IEqualityComparer<T>` to the SequenceEqual method.
- **Order Matters:** SequenceEqual considers the order of elements. If two sequences contain the same elements but in a different order, SequenceEqual will return false.
- **Performance:** SequenceEqual can be more efficient for large sequences than manually comparing elements because it will stop as soon as it finds two unequal elements.
- **Deferred Execution:** Like many LINQ methods, SequenceEqual uses deferred execution.

When to Use LINQ SequenceEqual Method in C#?

You should use the LINQ SequenceEqual method in C# when comparing two sequences (collections or arrays) to determine if they contain elements in the same order. Here are common scenarios where you should consider using the SequenceEqual method:

- **Data Validation:** When you want to validate that two collections or arrays have identical elements and order, especially when working with data that should match or have consistency.
- **Unit Testing:** In unit testing, you can use SequenceEqual to verify whether the actual output of a method or operation matches the expected output in terms of elements and their order.
- **Data Integrity Checks:** When dealing with data from different sources or databases, you can use SequenceEqual to ensure that data retrieved from one source matches the data retrieved from another source.
- **Sorting and Ordering:** If you have sorting or ordering logic in your code, SequenceEqual can be used to verify that the sorting or ordering process did not change the sequence's content.
- **Comparing User Input:** In applications that involve user input, you can use SequenceEqual to compare user-provided data with expected data to check if it matches, ensuring data consistency.
- **Testing Algorithm Outputs:** When working with algorithms or custom logic that generates sequences, you can use SequenceEqual to verify that the output of your algorithm matches the expected output.

- **Database Queries:** When you're querying a database and retrieving data as sequences, you can use `SequenceEqual` to compare the retrieved data against an expected dataset.
- **API Responses:** When interacting with APIs, `SequenceEqual` can help you validate that the data received from the API matches what you expected, allowing you to ensure data integrity.
- **Caching and Caching Validation:** In caching scenarios, you can compare data retrieved from a cache with data retrieved from a source to check if the cached data is still valid.
- **Monitoring and Data Consistency:** In real-time monitoring or data synchronization applications, `SequenceEqual` can be used to verify that data remains consistent between different systems or at different points in time.

The `SequenceEqual` method is valuable when you must compare two sequences to ensure they contain the same elements in order. It is particularly useful for data validation, unit testing, data integrity checks, and various scenarios where you need to verify the consistency and correctness of sequences.

In the next article, I will discuss the [LINQ Partitioning Methods in C#](#) with Examples. I explain the LINQ `SequenceEqual` Method in C# with Examples in this article. I hope you understand the need and use of the **LINQ `SequenceEqual` Method** in C#.